Cost-Aware Tool Usage for LLMs

Joshua Mayhugh & Shyam Ramachandran

Texas A&M Computer Science

Video Link:
`https://youtu.be/Ulxrt5P2ycE`
Repository Link:
`https://github.com/jmayhugh1/LLM_tool_training_RL/tree/main`

Abstract

This paper presents a reinforcement-learning framework for teaching pretrained language models to make more reliable and structured tool calls when interacting with external APIs. Our approach combines `Supervised Fine-Tuning` (SFT) on the `Qwen-2 0.5B` model with `Group Relative Policy Optimization` (GRPO), using a deterministic reward designed to reduce hallucinated tools or parameters, enforce the correct `<calls>` format, and discourage unnecessary text outside the tool-call block. After establishing a strong SFT baseline on the `xLAM function-calling` dataset, GRPO further refines the policy through KL-regularized updates, yielding a measurable reduction in evaluation loss and improved structural consistency in generated outputs. We also observe an increase in next-token Shannon entropy, indicating a shift toward a broader but still controlled policy distribution under RL training. Together, these results demonstrate that GRPO can improve tool-use reliability without requiring a value model or extensive hardware resources, offering a lightweight and scalable approach for building cost-aware tool-using LLMs.

Keywords: Reinforcement Learning, GRPO, Large Language Models, LoRA

1 Introduction

Large Language Models (LLMs) have improved their capabilities across a wide range of tasks, yet they still face limitations when relying solely on internal parameters. Recent research has focused on model tool-augmentation, but simply giving models access to tools isn't a complete solution. In many real-world deployments, external tool calls come with a drawback. Standard approaches like Supervised Fine-tuning (SFT) often lead to inefficient systems that can produce

wrongly formatted or hallucinated API calls, causing issues when those tools are utilized.

In this work, we present a framework for training cost-aware tool-using agents that is both effective and efficient on our testing methods. We focus on using SFT and GRPO on a baseline Qwen2-0.5B [1] model to refine the model's decision-making process without the need for an explicit actor-critic network. Our pipeline begins with SFT on the xLAM function-calling dataset [7], and then GRPO with a custom reward function designed to optimize the tool calling process. Through our combined methods, we provide a template for building scalable, efficient tool-calling agents.

2 Motivation

Reinforcement Learning (RL) has played an important role in fine-tuning large language models so they better align with human preferences. Models trained only on next-token prediction often generate text that is irrelevant or disconnected from what a user actually wants. RL-based methods have shown that they can meaningfully shift model behavior toward human-preferred outputs [2]. Large language models in particular have benefited from techniques such as Reinforcement Learning from Human Feedback (RLHF) [14], with algorithms like Proximal Policy Optimization (PPO) used to train the model toward those preference signals [8].

The success of RLHF shows that RL can be used to reliably align models with complicated human preferences, which is promising for fields where supervised learning alone is not enough.

Recent reasoning models are strong at many text-based tasks but still struggle in settings where problems are too difficult to solve with text-only reasoning. In these cases, giving the model the ability to use an external computational tool is much more effective [3]. Our goal is to combine the strengths of RL-based training with the advantages of tool use. We aim to fine-tune a model that reliably formats outputs and selects the correct tool APIs in a consistent and aligned way.

Our initial project proposal suggested using Proximal Policy Optimization, a common method for reinforcement learning with LLMs. However, recent advances in the field and the availability of training libraries motivated us to explore a more efficient and easy-to-use alternative. Group Relative Policy Optimization (GRPO) [12] addresses some of these limitations of PPO, including its need for a separate value model as well as a non-deterministic reward model class, both of which require additional memory and compute resources.

3 Related Work

3.1 Foundations of Tool-Assisted LLMS

Early approaches to tool usage relied mostly on self-supervised learning and fine-tuning. **Toolformer** started using inline <API> tags [10] to teach models when to call external tools. While it did demonstrate that API calls could be learned in a self-supervised manner, later works like **PAL** [5] improve this by combining natural language reasoning with the Python execution, improving performance by separating reasoning from program execution. More recently, **Gorilla** [6,9] integrates retrieval-aware training (RAT), improving these models to adapt to massive, constantly changing API documentation. Together, these methods establish a baseline for tool execution, but mainly focused on how to call these tools, rather than optimizing for when a tool was necessary.

3.2 Improving Tool Usage using RL

To move beyond static supervised baselines, recent research has applied reinforcement learning (RL) to optimize decision-making policies. For instance, **ReTool** [3] improves tool selection by rewriting reasoning traces into hybrid code-text formats, optimizing tool choice with outcome-based rewards. Other frameworks such as **AGILE** [4], **STEPTOOL** [16], and **Nemotron-Research-Tool-N1** [17] show that PPO- and GRPO-style training can shape multi-step reasoning, give feedback at each tool step, and let models discover better reasoning formats instead of copying pre-defined traces. Our work builds directly upon these insights, but designs the reward to care about using the right tool IDs, avoiding made-up tools, and keeping strict code friendly formatting.

3.3 Making RLHF Scalable

HybridFlow and the **TRL** [15] library have been valuable in making algorithms like PPO and GRPO accessible, with both streamlining the training pipeline. In particular, TRL's implementation of GRPO avoids the need for a separate value model, which our implementation uses on top of a QLoRA-tuned Qwen-2 0.5B baseline, reducing the memory overhead that is often encountered with basic hardware.

3.4 DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

GRPO is a modern reinforcement learning technique used for training Large Language models that improves upon PPO by dropping the need for a separate "value model". Each problem is treated as an episodic MDP, in which the policy (LLM) generates a full trajectory of tokens(actions represent the next-token choices) and the episode finishes when an answer is produced. For each prompt, the policy samples G candidate solutions, and a reward model assigns each

trajectory a scalar return. GRPO then computes a group-relative advantage for the trajectory:

$$A_i = \frac{r_i - \bar{r}_{\text{group}}}{s_{\text{group}} + \epsilon},$$

where r_i is the trajectory reward, $\bar{r}_{\text{group}}$ is the group mean, and s_{group} is the reward standard deviation. Policy updates follow this PPO style clipped objective,

$$L = \mathbb{E}[\min(\rho_i A_i,\ \text{clip}(\rho_i, 1-\epsilon, 1+\epsilon)\, A_i)]\,,$$

with likelihood ratio $\rho_i = \pi_\theta(y_i)/\pi_{\theta_{\text{old}}}(y_i)$. GRPO performs stable policy gradient RL without a critic, using group baselines to reduce variance while maintaining optimization behavior similar to PPO. [12]. This provides a clear and computationally efficient policy-gradient update rule, aligning our project with a reinforcement learning based fine tuning method.

4 Methodology

4.1 Model Selection and Supervised Fine-Tuning

We established our supervised baseline using **Qwen2-0.5B-Instruct**. We selected this model because of its ability to balance instruction-following capability with the efficiency required for rapid iteration. The base model was loaded in 4-bit precision using `BitsAndBytes`, with QLoRA adapters trained on the frozen parameters.

Dataset and prompt format. Supervised fine-tuning was conducted on the `Salesforce/xlam-function-calling-60k` dataset [7], which maps natural language queries to tool specifications and target function calls. The format of the data is as follows: each example is a JSON record containing (1) a `"query"` field with the user instruction, (2) a `"tools"` field storing a JSON-encoded list of available tools with their names, descriptions, and argument schemas, and (3) an `"answers"` field containing one or more JSON-encoded function-call outputs that represent valid solutions.

We formatted the data into prompt-completion pairs, where the prompt captures the user query and tool definitions,

$$\text{prompt} = \texttt{<user>}\ \text{query}\ \texttt{</user> <tools>}\ \text{tools}\ \texttt{</tools>}$$

and the completion gets the structured function call.

$$\text{completion} = \texttt{<calls>}\ \text{answers}\ \texttt{</calls> <eos>}.$$

The resulting dataset is split into train and test sets (default 90/10 split), with 30,000 examples sub-sampled for efficiency. This resulting format also explicitly teaches the model to map from a user query and a set of available tools to a structured `<calls>...</calls>` block, which later aligns with the GRPO reward.

QLoRA configuration. We applied LoRA adapters to all linear projection layers in the attention and feed-forward blocks (`k_proj`, `q_proj`, `v_proj`, `o_proj`) and feed-forward projections (`gate_proj`, `down_proj`, `up_proj`). The adapters were configured with rank $r = 16$, scaling factor $\alpha = 16$, and a dropout rate of 0.05, allowing for adaptation of the frozen base model without encountering any memory limitations.

Supervised fine-tuning setup. We conducted supervised fine-tuning (SFT) using the `SFTTrainer` from the TRL library. To ensure the model focuses on learning the tool calling process rather than mimicking the user queries, we set `completion_only_loss` to `True`. This confirms that only tokens in the `<calls>...</calls>` completion contribute to the loss, while the prompt is treated as conditioning context.

Optimization was perform using the (`adamw_8bit`) optimizer with gradient checkpointing, specifically chosen to maximize our batch size given our limited hardware. We trained for 200 steps with a learning rate of 1×10^{-4}, employing a linear decay schedule and a 10% warm up period to initially stabilize the LoRA parameters. We chose 200 steps empirically as that is when we noticed the training loss would reach its plateau. During this training process we tracked the token-level cross-entropy loss over the model's logits vs the target tokens, and we checked the models performance against the Test set every 100 steps to verify the model is generalizing to unseen data. We also track the Shannon entropy [11] of the next-token distribution, to determine how uncertain the next token prediction is.

The resulting model serves as the reference policy for the subsequent RL stage. While this SFT phase ensures that the model can reliably produce correct `<calls>` blocks, it does not inherently optimize for proper formatting, structure, and minimizing hallucinations. These properties of the tool-calling APIs are important because each tool exposes a fixed interface with strict formatting and input constraints. In the next stage, GRPO mitigates these limitations by optimizing the policy using a cost-aware reward function.

4.2 GRPO Fine-Tuning and MDP Structure

We view GRPO-based fine-tuning as a single-trajectory Markov decision process (MDP), where each training prompt forms one episode. Let x denote a prompt containing a `<user>...</user>` block with one or more IDs, and let $y = (a_1, \ldots, a_T)$ be the generated completion.

At token step t, the state is the text prefix

$$s_t = (x, y_{<t}),$$

which consists of the fixed prompt and all tokens generated so far. The action is the next token:

$$a_t \in \{1, \ldots, V\}$$

Transitions are deterministic:

$$P(s_{t+1} \mid s_t, a_t) = (x, y_{<t} \circ a_t),$$

An episode ends when the model emits an EOS token or reaches the maximum completion length.

Reward. The terminal reward $R(x, y)$ is a deterministic function of the concatenated text $t = x \| y$. It starts from zero, accumulates a set of additive contributions, and is then clipped to $[-1.0, 1.0]$. The reward is designed to capture three main aspects are the `<calls>` blocks well formed, consistent with the prompt, and do they fit the style/length requested.

First, we enforce the overall structure. If the completion is empty, we set $R = -1.0$. If no `<calls>...</calls>` block is present, we apply a penalty of -0.7. Extra text before or after the `<calls>` block incurs -0.2 each, and generating a `<response>` tag instead of tool calls incurs -0.3.

Next, we check if the contents of the block can be interpreted as a list of calls. If the block cannot be parsed as a list of Python-style dictionaries, we subtract 0.6 from the reward, if parsing succeeds and at least one dictionary is found, 0.6 is added. From the prompt's `<tools>` block we extract the set of allowed tools and their parameter names. Each hallucinated tool incurs a -0.2 penalty, and each hallucinated parameter incurs -0.1. If at least one call is present and no tools or parameters are hallucinated, we add a further $+0.2$.

Finally, we tie the calls back to the user IDs. Let U be the set of IDs in the `<user>` block and C the set of IDs referenced in `<calls>`. We compute:

$$r = \frac{|U \cap C|}{|U|}$$

and add a coverage bonus of $0.2r$, with an extra $+0.1$ when $r = 1$. Each hallucinated ID in $C \backslash U$ incurs -0.1. Then, we add small style and regularization terms: $+0.05$ if the calls block contains both "{" and "}", $+0.05$ if it contains the substring "`name`", and -0.1 if the completion exceeds 600 characters. After all terms are summed, we clamp

$$R(x, y) \in [-1.0, 1.0].$$

GRPO objective and training. For each prompt x, GRPO samples a group of $K = 8$ completions

$$\{y^{(1)}, \ldots, y^{(K)}\}$$

from the current policy π_θ. The corresponding rewards $\{R^{(k)}\}$ are computed using the rule above and normalized within the group to obtain advantages $A^{(k)}(x, y^{(k)})$. The policy is updated according to

$$\nabla_\theta J(\theta) = \mathbb{E}\left[A(x, y) \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] - \beta \nabla_\theta \mathrm{KL}\big(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\mathrm{SFT}}(\cdot \mid x)\big),$$

where π_{SFT} is the supervised baseline and $\beta = 0.05$ controls the strength of Kullback–Leibler (KL) regularization [13] The KL Divergence term measures how much the updated policy deviates from the SFT reference model, providing a stabilizing force during reinforcement learning. We chose $\beta = 0.05$ because it allowed the policy to explore reward improving behaviors while maintaining controlled divergence. We also had a reasonably capable SFT model (as discussed in Table 1), so we elected to choose a lower β to remain close to the reference model.

5 Results

This section contains Tables, Figures and an Analysis of the results.

Figure 1: Training Loss over 200 steps for SFT Trainer

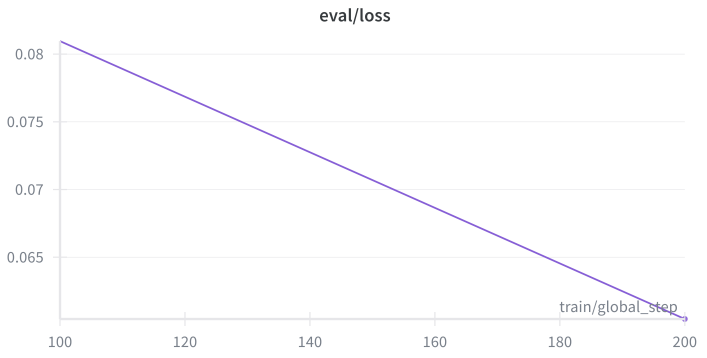

Figure 2: Evaluation Loss against test set, checked at 100 steps and 200 steps for SFT Trainer

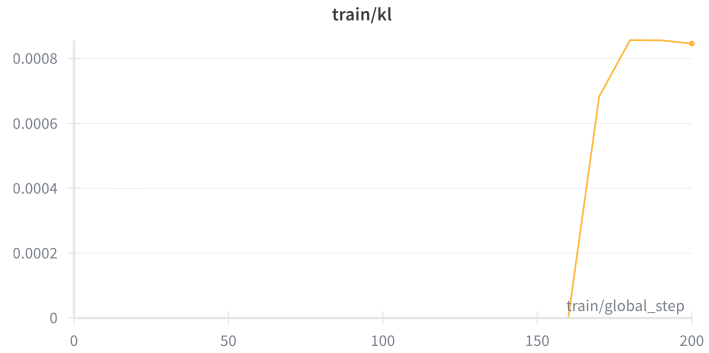

Figure 3: KL Divergence over 200 steps for GRPO Trainer

Method	Step	Training Loss	Test Loss	Entropy
SFT	200	0.063900	0.060436	1.038168
SFT-GRPO	200	N/A	0.000199	1.80603

Table 1: SFT Training Metrics at Selected Steps

5.1 Analysis

After the SFT step we received a quite capable model against this dataset, but we were not able to decrease the loss any further than the metrics given in Table 1. As we can see the training loss plateaus at around 175 steps as can be seen in Figure 1. However the additional training with GRPO improved our policy very moderately but gave us a far superior Test loss with 0.000199. It is worth noting that GRPO does not optimize cross-entropy loss, so we do not have that metric. But we did calculate the cross-entropy loss with respect to the test set after training. We can see in Figure 3, which plots the training steps against KL Divergence there is a noticeable increase in the divergence of the Policy Model from the base SFT model which is likely what led to our slight performance increase.

We also notice that the GRPO model has an increased Shannon Entropy, from 1.038168 to 1.80603. This indicates a shift in the deterministic behavior that is learned during SFT towards a more exploratory policy under GRPO. SFT minimizes cross-entropy loss and pushes the model to assign high probability to a single target completion, while our GRPO reward permits many valid tool call outputs. Because these alternatives receive similar reward, the policy does not need to collapse onto one specific sequence, leading to a broader next-token distribution.

5.2 Limitations

While our results show that GRPO can improve tool-use behavior, several limitations constrain how strongly and broadly they can be interpreted. Our evaluation relies primarily on cross-entropy loss rather than measures of tool use reliability, such as rates of malformed `<calls>` blocks, hallucinated tools, or missing arguments. These kinds of metrics would more directly reflect the behaviors the rewards signal is intended to encourage.

A second limitation is the domain specific nature of the dataset. The model was trained exclusively on the xLAM function-calling data set, so performance on broader, real world APIs remains untested. The reward function is also entirely hand-tuned and deterministic, so it only approximates true tool-use costs and may encourage reward hacking or other unintended shortcuts

Finally, our study uses a relatively small model (Qwen2-0.5B) and has short training runs, and the model's behavior on larger datasets was not explored. Future work should evaluate the robustness of GRPO under multiple seeds, larger models, richer reward signals, and more realistic tool usage.

6 Conclusion

We present a framework for training LLMS to better make better decisions about tool use. By combining SFT and GRPO, the model learns both the mechanics behind invoking tools, judgment for when these tools are worth calling, and the proper formatting to make the tool code friendly. The approach's reasonable hardware requirements and simple reward function make the model accessible for many, and can be adapted on for future LLM adaptations.

References

[1] "Qwen2 technical report," 2024.

[2] P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," 2023. [Online]. Available: https://arxiv.org/abs/1706.03741

[3] J. Feng, S. Huang, X. Qu, G. Zhang, Y. Qin, B. Zhong, C. Jiang, J. Chi, and W. Zhong, "Retool: Reinforcement learning for strategic tool use in llms," 2025. [Online]. Available: https://arxiv.org/abs/2504.11536

[4] P. Feng, Y. He, G. Huang, Y. Lin, H. Zhang, Y. Zhang, and H. Li, "Agile: A novel reinforcement learning framework of llm agents," 2024. [Online]. Available: https://arxiv.org/abs/2405.14751

[5] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, "Pal: Program-aided language models," 2023. [Online]. Available: https://arxiv.org/abs/2211.10435

[6] M. Li, Y. Zhao, B. Yu, F. Song, H. Li, H. Yu, Z. Li, F. Huang, and Y. Li, "Api-bank: A comprehensive benchmark for tool-augmented llms," 2023. [Online]. Available: https://arxiv.org/abs/2304.08244

[7] Z. Liu, T. Hoang, J. Zhang, M. Zhu, T. Lan, S. Kokane, J. Tan, W. Yao, Z. Liu, Y. Feng *et al.*, "Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets," *arXiv preprint arXiv:2406.18518*, 2024.

[8] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," 2022. [Online]. Available: https://arxiv.org/abs/2203.02155

[9] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive apis," 2023. [Online]. Available: https://arxiv.org/abs/2305.15334

[10] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

[11] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[12] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "Deepseekmath: Pushing the limits of mathematical reasoning in open language models," 2024. [Online]. Available: https://arxiv.org/abs/2402.03300

[13] J. Shlens, "Notes on kullback-leibler divergence and likelihood," 2014. [Online]. Available: https://arxiv.org/abs/1404.2000

[14] N. Stiennon, L. Ouyang, J. Wu, D. M. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. Christiano, "Learning to summarize from human feedback," 2022. [Online]. Available: https://arxiv.org/abs/2009.01325

[15] L. von Werra, Y. Belkada, L. Tunstall, E. Beeching, T. Thrush, N. Lambert, S. Huang, K. Rasul, and Q. Gallouédec, "Trl: Transformer reinforcement learning," https://github.com/huggingface/trl, 2020.

[16] Y. Yu, Z. Wang, W. Ma, Z. Guo, J. Zhan, S. Wang, C. Wu, Z. Guo, and M. Zhang, "Steptool: A step-grained reinforcement learning framework for tool learning in LLMs," 2025. [Online]. Available: https://openreview.net/forum?id=PNHjoWcQje

[17] S. Zhang, Y. Dong, J. Zhang, J. Kautz, B. Catanzaro, A. Tao, Q. Wu, Z. Yu, and G. Liu, "Nemotron-research-tool-n1: Exploring tool-using language models with reinforced reasoning," 2025. [Online]. Available: https://arxiv.org/abs/2505.00024